Preston-Check — Administrator Manual

For operators running the Preston-Check SaaS

2026-05-04

Administrator Manual

This manual covers what the operator does day-to-day to run the Preston-Check business: managing customers, issuing licenses, triaging support tickets, reviewing weekly threat-intel drafts, shipping releases. For incident-response procedures (rotate token, recover from compromise) see `docs/operator-runbook.md`. For the underlying surface-design rationale see `docs/portals-and-kpis.md`.

Daily checklist

The recommended ten-minute morning loop:

1. `https://admin.preston-check.com/` — check the customers page for anyone who churned overnight, the sales pipeline for new inbound, and the support inbox for P0/P1 tickets.
2. Triage any P0/P1 ticket within the SLA window (30 min for P0, 4h for P1). Reply via the ticket interface or email — the AI draft response is gated and never auto-sent.
3. Glance at the System health board. All nine tiles should be green. If anything's amber/red, read the underlying metric (latency, uptime, queue depth) and investigate.
4. If it's Monday, check the threat-intel triage queue — the weekly NVD ingest landed at 09:00 UTC.

The daily loop should fit in ten minutes if nothing is on fire. Most days it does.

Weekly checklist

The recommended thirty-minute Friday review:

1. **Customers** — sort by MRR; spot-check any customer whose MRR delta is non-zero (new, expansion, contraction, churn). Annotate the change reason in the customer detail panel.
2. **Sales pipeline** — review every card in Demo and Trial stages. Anything stalled > 14 days gets a check-in email or a downgrade to Qualified. Anything stalled > 30 days closes-lost.
3. **Threat-intel queue** — author grep patterns for any draft you haven't gotten to during the week. Any draft sitting in `proposed/` for > 4 weeks closes with a comment.
4. **Revenue dashboard** — verify MRR, ARR, NRR, and gross margin match Stripe's source-of-truth. Reconciliation drift > 1% means the webhook missed an event; investigate.
5. **Run the standard health check** (`docs/operator-runbook.md`) — five surfaces, expected codes 200/200/302/200/405.

Monthly checklist

Roughly two hours, end of month:

1. Cohort retention review — pull the heat-map from the Revenue page; identify any cohort with abnormal early-month decay.
2. Per-channel acquisition cost vs. LTV — pull the sales pipeline metrics by source channel; compare 90-day LTV vs. estimated acquisition spend per channel.
3. Catalog growth — count `community/proposed/` drafts that closed accepted vs. rejected; track the acceptance rate as a leading indicator of threat-intel pipeline quality.
4. AI quality — sample 20 random false-positive flags from the ai-cache directory; compare AI classification to your own; track precision/recall over time.
5. Annual State of Fintech Security report — if Q4, draft the year's narrative pulling from D1 telemetry data.

Quarterly checklist

Once per quarter, half-day deep dive:

1. Per-customer expansion conversations — Enterprise customers get a quarterly architecture review (a contractual deliverable).
2. Competitor scan — what have Snyk Code, Semgrep Pro, CodeQL, Trivy shipped this quarter? Where do we lag, where do we lead?
3. Strategy doc refresh — `docs/strategy/moat-strategy.md` and `docs/strategy/gold-standard-playbook.md` get an annotation pass marking shipped items, deferring others, adding new ones.

Issuing a license

License signing requires the operator's hardware-bound private key.

```
# On your local machine where ~/.preston-check/keys/private.pem lives:
./tools/issue-license.sh \
  --customer "helios-banking" \
  --tier enterprise \
  --expires 2027-04-12

# Output: ./out/helios-banking-2026.license
# Email the file to the customer; they install it at
# ~/.preston-check/license or pass via PRESTON_LICENSE env var.
```

The license is a PEM-style envelope with a JSON payload (customer ID, tier, expiry) signed by the Ed25519 private key. The runner's open- source `lib/license.sh` verifies the signature offline against the public key embedded at `lib/license_pubkey.pem`. If signature fails or expiry has passed, the runner falls back to the Free tier with a clear status note.

The future Admin Portal "License issuance" surface will provide a UI wrapper that calls this same script via Touch ID — until then, the operator runs it from the terminal.

Revoking a license

Three options:

1. **Wait for expiry** — soft revocation; the license stops working on the expiry date. Best for end-of-contract.
2. **Add to revocation list** — TBD; a future runner version will check a published revocation file at startup.
3. **Rotate the signing key** — hard revocation; invalidates every active license. Use only for compromise scenarios; see `docs/operator-runbook.md` "Compromised license signing key".

Handling support tickets

Support tickets arrive via three channels:

- **Email** to `support@preston-check.com` — Cloudflare Email Routing forwards to the operator's actual mailbox; ticketing software (TBD) ingests and creates a ticket.
- **Slack** in the customer's shared channel — slash-command bot creates a ticket from any thread.

- **Direct via the customer portal** — paying customers can submit tickets from the portal's settings page.

SLA tiers (informal until contracts spell them out):

PRIORITY	DEFINITION	RESPONSE	RESOLUTION
P0	Production outage; customer cannot scan	30 min	4h
P1	Production degraded; workaround available	4h	24h
P2	Bug, feature request, "how do I…"	24h	best effort

The metric to optimize is **time to first response** (TTFR) — that's what predicts NPS and retention better than time to resolution. The admin portal Support page tracks median TTFR as a KPI tile.

Triaging the threat-intel queue

Every Monday at 09:00 UTC, `tools/sync-threat-intel.py` opens a PR against the repo with new draft check files in `checks/community/proposed/`. Each draft is a candidate check auto-generated from a recent CVE that matches fintech-relevance keywords.

The triage flow:

1. Open the PR. Read each draft's `description` and `origin` metadata. The `origin` field cites the CVE ID and brief description.
2. For each draft, decide: **accept**, **reject**, or **defer**.
3. **Accept** — author the actual grep pattern in the check body (replacing the auto-generated stub). Move the file from `checks/community/proposed/` to `checks/community/accepted/`. The check ships in the next release.
4. **Reject** — close the draft with a comment explaining why (not fintech-relevant, duplicate of existing check, false- positive prone, etc.). The CVE is added to the `processed_cves` list so it won't be re-drafted.
5. **Defer** — leave the draft in `proposed/` for the week; revisit next Monday. Anything sitting > 4 weeks should close-rejected.

The acceptance rate is a moat-health KPI. Below 10% means the keyword list is too broad (lots of irrelevant CVEs surfacing). Above 50% means it's too narrow (we're missing things).

Releasing a new version

Standard semver. Patch (1.7.x) for bug fixes, minor (1.8.0) for new checks or features, major (2.0.0) reserved for breaking changes.

```
# 1. Bump PRESTON_VERSION in preston-check.sh
# 2. Update CHANGELOG.md with a new entry
# 3. Commit
git commit -m "vX.Y.Z: short summary"
# 4. Tag + push (release.yml takes over from here)
git tag -a vX.Y.Z -m "vX.Y.Z: summary"
git push origin master --tags
```

`.github/workflows/release.yml` automatically creates the GitHub Release with `install.sh` + tarball + sha256 sidecar. If the `HOMEBREW_TAP_TOKEN` secret is configured, the homebrew formula auto-updates too. Docker image push requires `DOCKERHUB_USERNAME` + `DOCKERHUB_TOKEN` secrets.

Adding a new framework or check

Framework integration is metadata-driven. To add coverage for a new framework:

1. Identify the controls the framework specifies that map to security patterns we already check.
2. For existing checks, add the framework citation to the `frameworks:` field in the check's PRESTON_META heredoc.
3. For new checks not yet in the catalog, write the check file following the conventions in `docs/architecture.md`. Place under `checks/community/proposed/` for self-review, then promote to `community/accepted/` after lint passes.
4. Update `docs/all-frameworks.md` and the README's framework table to reflect the new framework name.
5. Bump `PRESTON_VERSION`, write a changelog entry, tag a release.

The lint gate (`tools/lint-check.sh`) enforces metadata validity and the trust-tier path conventions. CI runs it automatically.

Customer onboarding flow

The intended end-state customer journey (production version still incomplete; current state mock-only):

1. Customer signs up at `https://app.preston-check.com/signup`

2. Magic-link email arrives, customer clicks
3. Onboarding wizard prompts: org name, primary language, repo URL
4. Customer installs the GitHub Action (the wizard generates the YAML snippet pre-filled with their license)
5. First scan runs; report appears in the customer portal
6. Stripe billing starts (free for OSS / 14-day trial / paid plan)

Until production is wired, manual onboarding looks like:

1. Customer emails `hello@preston-check.com` expressing interest
2. Operator (you) responds with a Pro license file via DM
3. Customer installs Preston-Check, points it at their repo, runs
4. Operator follows up after first week to gauge fit

Compliance and regulatory posture

The SaaS itself is governed by the disclosures in `SECURITY.md`. The Preston-Check brand and the "preston-check.com" trademark are owned by the operating LLC (per the anonymity strategy in `docs/strategy/anonymity-and-mystique.md`). The scanner code is Apache 2.0 forever; the SaaS layer (admin portal, customer portal, billing, license issuance) is proprietary.

Customer-data residency: by default, scan reports remain on the customer's machine. Telemetry that has been opted into is anonymized and stored in Cloudflare D1 (ENAM region) with a 90-day TTL on raw records and unlimited TTL on aggregates.

GDPR / CCPA stance: because telemetry contains no personally- identifiable information (only a salted SHA-256 hash of the repo URL), no data-subject access requests apply. Customers operating in the EU/UK/CA who want a data-processing agreement can request one through sales@preston-check.com.

Where things live

- **Public repo** — `https://github.com/preston-check/preston-check`
- **Customer-facing docs** — `docs/manuals/`, `docs/getting-started.md`
- **Operator-facing docs** — `docs/operator-runbook.md`, `docs/administrator-manual.md` (this file), `docs/portals-and-kpis.md`
- **Strategy** — `docs/strategy/`
- **Source** — `preston-check.sh`, `lib/`, `checks/`, `modules/`, `web/`, `workers/`, `tools/`

This file is the operator's daily-use admin reference. Update it as the SaaS surface grows.